

COMPUTE, STORAGE, NETWORKING AND THE DEPLOYMENT LANDSCAPE

Lab Guide

Six hands-on labs and two activities on Google Cloud — console or Cloud Shell only, no code to write.

Version 1.0 · September 2026 · gcloud 583.x · region asia-south1

Contents

How to use this guide	3
Lab 0 — Landing in the sandbox.....	4
Lab 1 — A VM, then a fleet	6
Lab 2 — Ship a container without a Dockerfile — then ship a function.....	10
Lab 3 — A catalogue bucket that cleans up after itself	13
Lab 4 — A private subnet that can still reach Google	16
Lab 5 — Ship the two-service app.....	20
Activities A and B (no keyboard)	24
Appendix — Environment and versions	26

How to use this guide

Every lab in this guide is done either in the Google Cloud console or with commands typed into Cloud Shell. You never write or edit application code: where a lab deploys an application, the application is supplied complete in the lab kit and your job is to build, store, deploy, secure and observe it.

Conventions

- Cloud Shell blocks (teal) are copy-pasteable. Run them one block at a time, in order.
- Blanks written as _____ in a starter script are the parts you fill in; the answers are in the expected panels here and on the debrief slides.
- Every step that has a console equivalent shows a Console path line: the navigation, the page and the button. Use whichever you prefer — the result is identical.
- Expected panels show the real output format. Names, IPs, timestamps and hashes will differ in your project.
- Angle-bracket placeholders such as <your-project-id> must be replaced with your own value. The scripts read the project id for you with `gcloud config get-value project`.
- Every lab that creates resources ends with a teardown step. A lab is finished when the resource list is empty — not when the page loads.

Getting the kit into Cloud Shell

Upload the zip through the Cloud Shell menu (: → Upload) or drag it onto the terminal, then:

```
$ unzip -q gcp-landscape-lab1-starter.zip && cd lab1 && ls
README.md lab1.sh startup.sh teardown.sh
```

Where to get help

- Raise a hand — the trainer pairs you with a neighbour rather than debugging in front of the room.
- Each lab ends with a Troubleshooting section covering the failures that actually happen.
- The solution kit for each lab is the completed command sheet; the trainer releases it after the debrief.

Cost note

Everything here runs inside a sandbox project the programme pays for. Idle resources still cost money overnight — three small VMs are roughly US\$1 a day, an idle GKE Autopilot cluster US\$2–3 a day. Tear down before every break.

Lab 0 — Landing in the sandbox

Estimated time	Difficulty	Environment	Kit
10 min	●○○	Cloud Shell, region asia-south1	none

OBJECTIVE

Cloud Shell open on your sandbox project, gcloud pointed at asia-south1, the six APIs enabled and the toolchain verified.

PREREQUISITES

- A Google account added to a sandbox project as Owner (sent as pre-work).
- A browser. Nothing to install.

Steps

Step 1 — Open Cloud Shell on your sandbox project

Console path: console.cloud.google.com → project picker (top bar) → your sandbox → Activate Cloud Shell icon (top right).

EXPECTED

```
Welcome to Cloud Shell! Type "help" to get started.
Your Cloud Platform project in this session is set to sandbox-rohan-07.
rohan@cloudshell:~ (sandbox-rohan-07) $
```

Step 2 — Point gcloud at the project, region and zone

CLOUD SHELL

```
gcloud config set project <your-project-id>
gcloud config set compute/region asia-south1
gcloud config set compute/zone asia-south1-a
gcloud config list
```

EXPECTED

```
Updated property [core/project].
Updated property [compute/region].
Updated property [compute/zone].
[compute]
region = asia-south1
zone = asia-south1-a
[core]
account = you@example.com
project = sandbox-rohan-07
```

Step 3 — Enable the six APIs used today

Console path: APIs & Services → Enable APIs and services → search each API → Enable (six times). The command does all six at once.

CLOUD SHELL

```
gcloud services enable compute.googleapis.com run.googleapis.com \
  cloudbuild.googleapis.com artifactregistry.googleapis.com \
  secretmanager.googleapis.com storage.googleapis.com
```

EXPECTED

```
Operation "operations/acat.p2-6..." finished successfully.
```

Takes 30–90 seconds. Re-running is harmless.

Step 4 — Verify the toolchain

CLOUD SHELL

```
gcloud --version | head -1
docker --version
python3 --version
kubectl version --client | head -1
```

EXPECTED

```
Google Cloud SDK 583.0.0
Docker version 28.x
Python 3.13.5
Client Version: v1.33.x
```

CHECK YOURSELF

- The prompt ends with (your-project-id).
- `gcloud config list` shows `asia-south1 / asia-south1-a`.
- `gcloud services list --enabled | grep -c googleapis` prints a number ≥ 6 .

Troubleshooting

Symptom	Cause	Fix
"You do not currently have an active account selected"	Cloud Shell opened in the wrong Google account	Switch account (avatar, top right) and reopen Cloud Shell
Enable fails with <code>PERMISSION_DENIED</code>	You are not Owner on this project	Tell the trainer — you get the spare project
Cloud Shell stuck on "Provisioning"	Session start-up hiccup	Reload the browser tab; if it persists, pair up
Prompt shows another project	Several sandboxes in the picker	<code>gcloud config set project <your-project-id></code>

Summary

You now have one project, one region and six enabled APIs. Every later lab assumes exactly this state — if you switch machines or accounts, redo steps 2 and 3.

Lab 1 — A VM, then a fleet

Estimated time	Difficulty	Environment	Kit
20 min	●●○	Cloud Shell, region asia-south1	gcp-landscape-lab1-starter.zip (solution: -solution.zip)

OBJECTIVE

One Compute Engine VM serving a page from a startup script, then an instance template, a two-VM managed instance group and an autoscaler — and everything torn down.

PREREQUISITES

- Lab 0 completed.
- Kit unzipped: lab1.sh (three blanks), startup.sh, teardown.sh.

Steps

Step 1 — Create one VM from the debian-12 image family with the startup script

Console path: Compute Engine → VM instances → Create instance → name bookshelf-vm, region asia-south1, zone a, e2-micro, Boot disk: Debian 12, Networking: tag http-server, Advanced → Automation: paste startup.sh.

CLOUD SHELL

```
gcloud compute instances create bookshelf-vm \
  --machine-type=e2-micro \
  --image-family=debian-12 --image-project=debian-cloud \
  --boot-disk-size=10GB --boot-disk-type=pd-balanced \
  --tags=http-server \
  --metadata-from-file=startup-script=startup.sh
```

EXPECTED

```
Created [https://www.googleapis.com/compute/v1/projects/sandbox-rohan-07/zones/asia-south1-a/instances/bookshelf-vm].
NAME          ZONE          MACHINE_TYPE  PREEMPTIBLE  INTERNAL_IP  EXTERNAL_IP
STATUS
bookshelf-vm  asia-south1-a  e2-micro      10.160.0.2   34.93.118.204
RUNNING
```

Step 2 — Allow HTTP to anything tagged http-server

Console path: VPC network → Firewall → Create firewall rule → name allow-http-80, network default, ingress, allow, targets: specified tags http-server, source 0.0.0.0/0, tcp:80.

CLOUD SHELL

```
gcloud compute firewall-rules create allow-http-80 \
  --network=default --direction=INGRESS --action=ALLOW \
  --rules=tcp:80 --source-ranges=0.0.0.0/0 --target-tags=http-server
```

EXPECTED

```
Creating firewall...done.
NAME          NETWORK  DIRECTION  PRIORITY  ALLOW  DENY  DISABLED
allow-http-80  default  INGRESS    1000      tcp:80  False
```

Step 3 — Look at the VM and open its page

CLOUD SHELL

```
gcloud compute instances list
gcloud compute instances describe bookshelf-vm \
  --format="value(networkInterfaces[0].accessConfigs[0].natIP) "
```

EXPECTED

34.93.118.204

Open `http://<that IP>` in a browser tab. Within ~60 s of boot you should see "Bookshelf baseline — Served by VM: bookshelf-vm".

CHECK YOURSELF

- The page loads and names bookshelf-vm.
- If it does not: wait a minute (`apt-get` is installing python3), then check the tag on the VM matches the firewall rule.

Step 4 — Fill blank 1.4 — an instance template with the same recipe, e2-small

Console path: Compute Engine → Instance templates → Create instance template → same choices as the VM, machine type e2-small.

CLOUD SHELL

```
gcloud compute instance-templates create bookshelf-tmpl \
  --machine-type=e2-small \
  --image-family=debian-12 --image-project=debian-cloud \
  --tags=http-server \
  --metadata-from-file=startup-script=startup.sh
```

EXPECTED

```
Created [https://www.googleapis.com/compute/v1/projects/sandbox-rohan-07/global/instanceTemplates/bookshelf-tmpl].
NAME                MACHINE_TYPE  PREEMPTIBLE  CREATION_TIMESTAMP
bookshelf-tmpl      e2-small      2026-09-06T10:31:42.117+05:30
```

Reference the image family, never a dated image name — next month's VMs get this month's patches for free.

Step 5 — Fill blank 1.5 — a zonal managed instance group of 2

Console path: Compute Engine → Instance groups → Create instance group → New managed instance group (stateless) → template bookshelf-tmpl, single zone asia-south1-a, number of instances 2.

CLOUD SHELL

```
gcloud compute instance-groups managed create bookshelf-mig \
  --template=bookshelf-tmpl --size=2 --zone=asia-south1-a
```

EXPECTED

```
Created [https://www.googleapis.com/compute/v1/projects/sandbox-rohan-07/zones/asia-south1-a/instanceGroupManagers/bookshelf-mig].
NAME                LOCATION      SCOPE  BASE_INSTANCE_NAME  SIZE  TARGET_SIZE
bookshelf-mig      asia-south1-a  zone   bookshelf-mig       0     2           bookshelf-
tmpl               no
```

Step 6 — Fill blank 1.6 — autoscale 1..3 on 60 % CPU with a 60 s cooldown

Console path: Instance groups → bookshelf-mig → Edit → Autoscaling: On, min 1, max 3, signal CPU utilisation 60 %, cool-down 60 s.

CLOUD SHELL

```
gcloud compute instance-groups managed set-autoscaling bookshelf-mig \
  --zone=asia-south1-a --min-num-replicas=1 --max-num-replicas=3 \
  --target-cpu-utilization=0.6 --cool-down-period=60
```

EXPECTED

```
Created [https://www.googleapis.com/compute/v1/projects/sandbox-rohan-07/zones/asia-south1-a/autoscalers/bookshelf-mig-xkq2].
---
autoscalingPolicy:
  coolDownPeriodSec: 60
  cpuUtilization: {utilizationTarget: 0.6}
  maxNumReplicas: 3
  minNumReplicas: 1
  mode: ON
```

Step 7 — Watch the group

Console path: Instance groups → bookshelf-mig → Monitoring tab shows the autoscaler decisions.

CLOUD SHELL

```
gcloud compute instance-groups managed list-instances bookshelf-mig --zone=asia-south1-a
```

EXPECTED

NAME	ZONE	STATUS	ACTION	INSTANCE_TEMPLATE
bookshelf-mig-4k7p	asia-south1-a	RUNNING	NONE	bookshelf-tmpl
bookshelf-mig-x2mc	asia-south1-a	RUNNING	NONE	bookshelf-tmpl
# about 10 minutes later, VMs idle, min=1:				
bookshelf-mig-4k7p	asia-south1-a	RUNNING	NONE	bookshelf-tmpl

Predict before you look: two idle VMs, minimum 1 — the autoscaler will shrink the group to one after its stabilisation window.

Step 8 — Tear down

CLOUD SHELL

```
bash teardown.sh
gcloud compute instances list # must be empty
```

EXPECTED

```
Lab 1 resources removed.
Listed 0 items.
```

CHECK YOURSELF

- list-instances showed two RUNNING VMs from bookshelf-tmpl.
- The autoscaler shows mode ON with min 1 / max 3.
- After teardown, instances list is empty and the firewall rule is gone.

Troubleshooting

Symptom	Cause	Fix
Page never loads	Firewall tag mismatch, or startup script still running	Check --tags=http-server on the VM; wait 60 s; gcloud compute ssh bookshelf-vm --command='tail /var/log/bookshelf.log'
QUOTA_EXCEEDED on create	Sandbox CPU quota	Use e2-small everywhere and size 1 for the MIG
set-autoscaling: "not found"	Zone flag missing or wrong	Always pass --zone=asia-south1-a for zonal groups

Symptom	Cause	Fix
Subnet/firewall delete fails in teardown	Order — VMs first	Re-run teardown.sh; it is idempotent

Challenge (fast finishers)

Make the group regional: recreate it with `--region=asia-south1` instead of `--zone` and list where the VMs landed. Then attach a health check (`gcloud compute health-checks create http hc-80 --port=80`) and enable autohealing with `update --health-check=hc-80 --initial-delay=120`.

Summary

You built the left edge of the compute spectrum: a VM from an image family, a reusable template, a managed group that replaces unhealthy members, and an autoscaler that changed the group size on its own. The concept it demonstrates: never build the second VM by hand.

Lab 2 — Ship a container without a Dockerfile — then ship a function

Estimated time	Difficulty	Environment	Kit
20 min	●●○	Cloud Shell, region asia-south1	gcp-landscape-lab2-starter.zip (solution: -solution.zip)

OBJECTIVE

The supplied bookshelf-api service running on Cloud Run with two revisions and a 50/50 traffic split, plus an HTTP function deployed with Cloud Run functions.

PREREQUISITES

- Lab 0 completed (run, cloudbuild and artifactregistry APIs enabled).
- Kit unzipped: bookshelf-api/ (complete — nothing to edit), hello-function/ (complete), lab2.sh with four blanks, teardown.sh.

The application is supplied finished. You do not open app.py or main.py except to read them. Everything you do is a gcloud flag or a console setting.

Steps

Step 1 — Fill blank 2.1 — deploy the API from source, public

Run this from inside the bookshelf-api folder. There is no Dockerfile: Google Cloud buildpacks detect Python from requirements.txt and read the start command from the Procfile.

Console path: Cloud Run → Deploy container → Service. The console deploys from a container image or a Git repository; for a folder in Cloud Shell use the command.

CLOUD SHELL

```
cd bookshelf-api
gcloud config set run/region asia-south1
gcloud run deploy bookshelf-api --source . --allow-unauthenticated
```

EXPECTED

```
Deploying from source requires an Artifact Registry Docker repository to store built containers.
A repository named [cloud-run-source-deploy] in region [asia-south1] will be created.
Do you want to continue (Y/n)? y
✓ Building and deploying new service... Done.
Service [bookshelf-api] revision [bookshelf-api-00001-k7q] has been deployed and is
serving 100 percent of traffic.
Service URL: https://bookshelf-api-482915730261.asia-south1.run.app
```

First deploy: 2–4 minutes (the build). Answer y to the repository prompt.

Step 2 — Call it

CLOUD SHELL

```
API_URL=$(gcloud run services describe bookshelf-api --format='value(status.url)')
curl -s $API_URL/ ; echo
curl -s $API_URL/books/2 ; echo
```

EXPECTED

```
{"books":4,"service":"bookshelf-api","version":"v1"}
{"author":"Kleppmann","id":2,"title":"Designing Data-Intensive
Applications","year":2017}
```

Step 3 — Fill blank 2.2 — a second revision with a new version label

Same source, one environment variable — enough to create a new, immutable revision.

Console path: Cloud Run → bookshelf-api → Edit & deploy new revision → Variables & secrets → add APP_VERSION = v2 → Deploy.

CLOUD SHELL

```
gcloud run deploy bookshelf-api --source . --set-env-vars APP_VERSION=v2
gcloud run revisions list --service bookshelf-api
```

EXPECTED

Service [bookshelf-api] revision [bookshelf-api-00002-p2m] has been deployed and is serving 100 percent of traffic.

REVISION	ACTIVE	SERVICE	DEPLOYED	DEPLOYED
BY				
✓ bookshelf-api-00002-p2m	yes	bookshelf-api	2026-09-06 11:02:14 UTC	
you@example.com				
✓ bookshelf-api-00001-k7q		bookshelf-api	2026-09-06 10:58:03 UTC	
you@example.com				

Step 4 — Fill blank 2.3 — split traffic 50/50 and call it six times

Console path: Cloud Run → bookshelf-api → Revisions → Manage traffic → set 50 % on each revision → Save.

CLOUD SHELL

```
LATEST=$(gcloud run revisions list --service bookshelf-api --format='value(name)' --
sort-by=~creationTimestamp --limit=1)
PREV=$(gcloud run revisions list --service bookshelf-api --format='value(name)' --
sort-by=~creationTimestamp --limit=2 | tail -n1)
gcloud run services update-traffic bookshelf-api --to-revisions=$LATEST=50,$PREV=50
for i in 1 2 3 4 5 6; do curl -s $API_URL/ ; echo; done
gcloud run services update-traffic bookshelf-api --to-latest
```

EXPECTED

Traffic:

```
50% bookshelf-api-00001-k7q
50% bookshelf-api-00002-p2m
{"books":4,"service":"bookshelf-api","version":"v1"}
{"books":4,"service":"bookshelf-api","version":"v2"}
{"books":4,"service":"bookshelf-api","version":"v2"}
{"books":4,"service":"bookshelf-api","version":"v1"}
{"books":4,"service":"bookshelf-api","version":"v2"}
{"books":4,"service":"bookshelf-api","version":"v1"}
{"books":4,"service":"bookshelf-api","version":"v2"}
{"books":4,"service":"bookshelf-api","version":"v1"}
```

Routing is per request and weighted, not sticky — you may see 4/2 rather than 3/3.

CHECK YOURSELF

- Two revisions listed, both active during the split.
- The six curls show both v1 and v2.
- After --to-latest, every call answers v2.

Step 5 — Fill blank 2.4 — deploy the HTTP function

Run from inside hello-function. The entry point is the Python function named hello in main.py; the base image is Google's maintained Python 3.13 runtime.

Console path: Cloud Run → Write a function → name hello-function, region asia-south1, runtime Python 3.13, allow unauthenticated → paste main.py and requirements.txt → entry point hello → Deploy.

CLOUD SHELL

```
cd ../hello-function
gcloud run deploy hello-function --source . --function hello --base-image python313 \
--allow-unauthenticated
```

```
FN_URL=$(gcloud run services describe hello-function --format='value(status.url)')
curl -s "$FN_URL/?name=bookshelf" ; echo
```

EXPECTED

```
Service [hello-function] revision [hello-function-00001-d4x] has been deployed and is
serving 100 percent of traffic.
Service URL: https://hello-function-482915730261.asia-south1.run.app
{"from":"cloud-run-function","message":"hello, bookshelf"}
```

Step 6 — Tear down**CLOUD SHELL**

```
cd .. && bash teardown.sh
gcloud run services list # must be empty
```

EXPECTED

```
Lab 2 services removed. (The cloud-run-source-deploy repo in Artifact Registry stays;
Lab 5 teardown removes it.)
Listed 0 items.
```

Troubleshooting

Symptom	Cause	Fix
"could not detect language" / build fails	You ran deploy outside bookshelf-api/	cd bookshelf-api and re-run
PERMISSION_DENIED on Cloud Build	APIs from Lab 0 not enabled	Re-run Lab 0 step 3
update-traffic: revision not found	Typed a revision name by hand	Use the \$LATEST / \$PREV lines from the script
Function returns 404	Wrong --function entry point	It must match the Python function name: hello

Challenge (fast finishers)

Give the second revision its own test URL before it takes traffic: `gcloud run deploy bookshelf-api --source . --set-env-vars APP_VERSION=v3 --no-traffic --tag canary`, then `curl` the URL printed with the tag prefix. Promote it with `update-traffic --to-tags canary=100`.

Summary

From a folder of source to a public HTTPS URL in one command; every deploy an immutable revision; rollout and rollback as a routing change; a function that is just a Cloud Run service with the container work removed. Nobody edited a line of code.

Lab 3 — A catalogue bucket that cleans up after itself

Estimated time	Difficulty	Environment	Kit
15 min	●●○	Cloud Shell, region asia-south1	gcp-landscape-lab3-starter.zip (solution: -solution.zip)

OBJECTIVE

A regional Standard bucket holding the catalogue exports, with lifecycle rules, versioning and an (unlocked) retention policy — then removed.

PREREQUISITES

- Lab 0 completed.
- Kit unzipped: docs/ (three sample exports), lifecycle.json with three blanks, lab3.sh, teardown.sh.

Steps

Step 1 — Create the bucket: regional, Standard, uniform bucket-level access

Bucket names are global across Google Cloud; the script appends your project id to keep yours unique.

Console path: Cloud Storage → Buckets → Create → name bookshelf-docs-<your-project-id> → Region asia-south1 → Standard → Uniform access → Create.

CLOUD SHELL

```
PROJECT=$(gcloud config get-value project)
BUCKET=gs://bookshelf-docs-$PROJECT
gcloud storage buckets create $BUCKET --location=asia-south1 \
  --default-storage-class=STANDARD --uniform-bucket-level-access
```

EXPECTED

```
Creating gs://bookshelf-docs-sandbox-rohan-07/...
```

Step 2 — Upload the exports and list them

Console path: Bucket → Upload files → select docs/*.txt (put them in a folder named exports).

CLOUD SHELL

```
gcloud storage cp docs/*.txt $BUCKET/exports/
gcloud storage ls -l $BUCKET/exports/
```

EXPECTED

```
Copying file://docs/catalogue-1.txt to gs://bookshelf-docs-sandbox-rohan-07/exports/catalogue-1.txt
...
 231  2026-09-06T05:31:12Z  gs://bookshelf-docs-sandbox-rohan-07/exports/catalogue-1.txt
 231  2026-09-06T05:31:12Z  gs://bookshelf-docs-sandbox-rohan-07/exports/catalogue-2.txt
 231  2026-09-06T05:31:13Z  gs://bookshelf-docs-sandbox-rohan-07/exports/catalogue-3.txt
```

Step 3 — Fill the three blanks in lifecycle.json, validate, apply

Rule 1: Standard objects move to NEARLINE after 30 days. Rule 2: an old version is deleted once 3 newer versions exist. Rule 3: everything is deleted at 365 days.

Console path: Bucket → Lifecycle tab → Add a rule (three times) with the same action/condition pairs.

CLOUD SHELL

```
python3 -m json.tool lifecycle.json > /dev/null && echo valid
```

```
gcloud storage buckets update $BUCKET --lifecycle-file=lifecycle.json
gcloud storage buckets describe $BUCKET --format="yaml(lifecycle_config)"
```

EXPECTED

```
valid
Updating gs://bookshelf-docs-sandbox-rohan-07/...
lifecycle_config:
  rule:
    - action: {storageClass: NEARLINE, type: SetStorageClass}
      condition: {age: 30, matchesStorageClass: [STANDARD]}
    - action: {type: Delete}
      condition: {isLive: false, numNewerVersions: 3}
    - action: {type: Delete}
      condition: {age: 365}
```

Answers: "NEARLINE", 3, 365. Rules evaluate about once a day; transitions are asynchronous.

Step 4 — Turn on versioning, overwrite a file, list the generations

Console path: Bucket → Protection tab → Object versioning: On.

CLOUD SHELL

```
gcloud storage buckets update $BUCKET --versioning
echo "id,title,author,year" > docs/catalogue-1.txt
gcloud storage cp docs/catalogue-1.txt $BUCKET/exports/catalogue-1.txt
gcloud storage ls -a $BUCKET/exports/catalogue-1.txt
```

EXPECTED

```
gs://bookshelf-docs-sandbox-rohan-07/exports/catalogue-1.txt#1757145601234567
gs://bookshelf-docs-sandbox-rohan-07/exports/catalogue-1.txt#1757145688765432
```

CHECK YOURSELF

- describe shows three lifecycle rules.
- ls -a shows two generations of catalogue-1.txt (the # suffix is the generation number).

Step 5 — Set a 1-day retention policy, then try to delete an object

The delete is expected to fail — that is the feature. The policy is NOT locked, so teardown can clear it. Never lock a retention policy on a sandbox bucket.

Console path: Bucket → Protection tab → Retention policy → Set → 1 day (do NOT lock).

CLOUD SHELL

```
gcloud storage buckets update $BUCKET --retention-period=P1D
gcloud storage rm $BUCKET/exports/catalogue-2.txt || echo "--> blocked by retention, as expected"
```

EXPECTED

```
ERROR: (gcloud.storage.rm) HTTPError 403: Object 'bookshelf-docs-sandbox-rohan-07/exports/catalogue-2.txt'
is subject to bucket's retention policy and cannot be deleted, overwritten or
archived until 2026-09-07T...
--> blocked by retention, as expected
```

Step 6 — Move one object to ARCHIVE by hand

This is what the lifecycle rule would do for you later; doing it once shows the class is per object.

Console path: Object → Edit storage class → Archive.

CLOUD SHELL

```
gcloud storage objects update $BUCKET/exports/catalogue-3.txt --storage-class=ARCHIVE
gcloud storage ls -L $BUCKET/exports/catalogue-3.txt | grep -i "storage class"
```

EXPECTED

```
Storage Class:  ARCHIVE
```

Step 7 — Tear down

Tear down clears the (unlocked) retention policy first, then deletes every version, then the bucket.

CLOUD SHELL

```
bash teardown.sh
gcloud storage ls    # your bucket must be gone
```

EXPECTED

```
Lab 3 bucket removed.
```

Troubleshooting

Symptom	Cause	Fix
409 "already exists" on create	Bucket names are global; the project-id suffix was lost	Check BUCKET=gs://bookshelf-docs-\$PROJECT printed your project id
lifecycle-file rejected	JSON typo (quotes, trailing comma)	python3 -m json.tool lifecycle.json shows the line
Tear down says retention policy still active	Policy was locked	It cannot be undone — leave the bucket; it expires in 1 day
rm succeeded in step 5	Retention step skipped or ran after rm	Re-run step 5 in order

Challenge (fast finishers)

Enable Autoclass on a fresh bucket (`gcloud storage buckets create gs://bookshelf-auto-$PROJECT --location=asia-south1 --enable-autoclass --uniform-bucket-level-access`) and compare its describe output with the rule-based bucket. Which would you choose for the catalogue exports, and why? Delete it afterwards.

Summary

Class, lifecycle, versioning, retention — four settings, and the bucket now enforces the data policy you used to write in a wiki. The one thing to remember: a locked retention policy is permanent.

Lab 4 — A private subnet that can still reach Google

Estimated time	Difficulty	Environment	Kit
20 min	...	Cloud Shell, region asia-south1	gcp-landscape-lab4-starter.zip (solution: -solution.zip)

OBJECTIVE

A custom-mode VPC with one planned subnet, an SSH-from-IAP-only firewall rule, a VM with no external IP, and Private Google Access proven to work — then removed in the right order.

PREREQUISITES

- Lab 0 completed.
- Kit unzipped: lab4.sh with four blanks, teardown.sh.
- First `gcloud compute ssh` in Cloud Shell generates an SSH key: accept the prompts (empty passphrase is fine in a sandbox).

Steps

Step 1 — Create a custom-mode VPC

Console path: VPC network → VPC networks → Create VPC network → name bookshelf-vpc → Subnet creation mode: Custom → Create (no subnets yet).

CLOUD SHELL

```
gcloud compute networks create bookshelf-vpc --subnet-mode=custom
```

EXPECTED

```
Created [https://www.googleapis.com/compute/v1/projects/sandbox-rohan-07/global/networks/bookshelf-vpc].
NAME          SUBNET_MODE  BGP_ROUTING_MODE  IPV4_RANGE  GATEWAY_IPV4
bookshelf-vpc  CUSTOM      REGIONAL          10.10.0.0/24
```

Step 2 — Fill blank 4.2 — one subnet with a /24 inside 10.0.0.0/8

A /24 is 256 addresses; Google reserves four (network, gateway, second-to-last, broadcast) so 252 are usable.

Console path: bookshelf-vpc → Subnets → Add subnet → name bookshelf-private, region asia-south1, IPv4 range 10.10.0.0/24.

CLOUD SHELL

```
gcloud compute networks subnets create bookshelf-private \
  --network=bookshelf-vpc --region=asia-south1 --range=10.10.0.0/24
```

EXPECTED

```
Created [https://www.googleapis.com/compute/v1/projects/sandbox-rohan-07/regions/asia-south1/subnetworks/bookshelf-private].
NAME          REGION      NETWORK          RANGE          STACK_TYPE
bookshelf-private  asia-south1  bookshelf-vpc    10.10.0.0/24   IPV4_ONLY
```

Step 3 — Fill blank 4.3 — SSH only from Google's IAP TCP-forwarding range

35.235.240.0/20 is the fixed range Identity-Aware Proxy uses for TCP forwarding. Nothing else is open on this network.

Console path: VPC network → Firewall → Create firewall rule → name bookshelf-allow-iap-ssh, network bookshelf-vpc, ingress, allow, target tag private-vm, source 35.235.240.0/20, tcp:22.

CLOUD SHELL

```
gcloud compute firewall-rules create bookshelf-allow-iap-ssh \
```

```
--network=bookshelf-vpc --direction=INGRESS --action=ALLOW \
--rules=tcp:22 --source-ranges=35.235.240.0/20 --target-tags=private-vm
```

EXPECTED

Creating firewall...done.

NAME	NETWORK	DIRECTION	PRIORITY	ALLOW	DENY	DISABLED
bookshelf-allow-iap-ssh	bookshelf-vpc	INGRESS	1000	tcp:22		False

Step 4 — Look at the routes the VPC got for free

Console path: VPC network → Routes → filter by network bookshelf-vpc.

CLOUD SHELL

```
gcloud compute routes list --filter="network:bookshelf-vpc"
```

EXPECTED

NAME	NETWORK	DEST_RANGE	NEXT_HOP
default-route-3f1a9c0b2e7d4a61	bookshelf-vpc	10.10.0.0/24	bookshelf-vpc
default-route-9b8e2d4c6a1f0e37	bookshelf-vpc	0.0.0.0/0	default-internet-gateway

Step 5 — Fill blank 4.5 — a VM with NO external IP

Console path: Compute Engine → Create instance → e2-micro, Debian 12 → Networking: tag private-vm, network interface bookshelf-vpc / bookshelf-private, External IPv4 address: None.

CLOUD SHELL

```
gcloud compute instances create private-vm --zone=asia-south1-a \
--machine-type=e2-micro --image-family=debian-12 --image-project=debian-cloud \
--subnet=bookshelf-private --no-address --tags=private-vm
```

EXPECTED

NAME	ZONE	MACHINE_TYPE	PREEMPTIBLE	INTERNAL_IP	EXTERNAL_IP	STATUS
private-vm	asia-south1-a	e2-micro		10.10.0.2		RUNNING

Step 6 — SSH in through IAP and prove nothing is reachable (yet)

Both curls time out after 5 seconds. That is the expected result: no external IP means no way out.

Console path: Compute Engine → VM instances → private-vm → SSH (the console button also tunnels through IAP).

CLOUD SHELL

```
gcloud compute ssh private-vm --zone=asia-south1-a --tunnel-through-iap \
--command='curl -s -m 5 -o /dev/null -w "internet: HTTP %{http_code}\n"
https://example.com || echo "internet: unreachable (expected)"; curl -s -m 5 -o
/dev/null -w "storage api: HTTP %{http_code}\n" https://storage.googleapis.com || echo
"storage api: unreachable (expected)";'
```

EXPECTED

```
internet: unreachable (expected)
storage api: unreachable (expected)
```

CHECK YOURSELF

- The VM has an INTERNAL_IP and an empty EXTERNAL_IP column.
- SSH through IAP works (the firewall rule is what allows it).

- Both curls print unreachable.

Step 7 — Fill blank 4.7 — turn on Private Google Access

Console path: VPC network → bookshelf-vpc → Subnets → bookshelf-private → Edit → Private Google Access: On → Save.

CLOUD SHELL

```
gcloud compute networks subnets update bookshelf-private --region=asia-south1 \
--enable-private-ip-google-access
```

EXPECTED

```
Updated [https://www.googleapis.com/compute/v1/projects/sandbox-rohan-07/regions/asia-south1/subnetworks/bookshelf-private].
```

Step 8 — Prove Google APIs now work and the internet still does not

HTTP 400 from storage.googleapis.com is success: the API answered (400 just means the request named no bucket).

CLOUD SHELL

```
gcloud compute ssh private-vm --zone=asia-south1-a --tunnel-through-iap \
--command='curl -s -m 5 -o /dev/null -w "internet: HTTP %{http_code}\n"
https://example.com || echo "internet: unreachable (expected)"; curl -s -m 5 -o
/dev/null -w "storage api: HTTP %{http_code}\n" https://storage.googleapis.com; gcloud
storage ls 2>&1 | head -3'
```

EXPECTED

```
internet: unreachable (expected)
storage api: HTTP 400
gs://bookshelf-docs-sandbox-rohan-07/
```

If the storage line still says unreachable, wait 30 seconds and re-run — the subnet change takes a few seconds to propagate.

Step 9 — Tear down — in order

VM first, then the firewall rule, then the subnet, then the network. A subnet with a VM in it cannot be deleted.

CLOUD SHELL

```
bash teardown.sh
gcloud compute networks list # only 'default' remains
```

EXPECTED

```
Lab 4 network removed.
NAME      SUBNET_MODE  BGP_ROUTING_MODE  IPV4_RANGE  GATEWAY_IPV4
default   AUTO         REGIONAL
```

Troubleshooting

Symptom	Cause	Fix
gcloud compute ssh hangs	IAP firewall rule missing or wrong source range	Rule must allow tcp:22 from 35.235.240.0/20 to tag private-vm
"Permission denied" on SSH	Key generation prompt was cancelled	Re-run; accept the key prompts
Step 8 storage line still unreachable	PGA not yet propagated, or wrong subnet	Wait 30 s; check subnets describe shows privateIpGoogleAccess: true

Symptom	Cause	Fix
Subnet delete fails	VM still exists	teardown.sh deletes in order; re-run it

Challenge (fast finishers)

Give the VM internet access without an external IP: create a Cloud Router (gcloud compute routers create nat-router --network=bookshelf-vpc --region=asia-south1) and Cloud NAT (gcloud compute routers nats create nat-config --router=nat-router --region=asia-south1 --auto-allocate-nat-external-ips --nat-all-subnet-ip-ranges), then re-run step 8 and watch the internet line change. Delete both before teardown.

Summary

Custom VPC, planned range, narrow firewall, no external IP, Private Google Access: that is the shape of every production subnet. The best news of the lab was an HTTP 400 — it proved the private path works.

Lab 5 — Ship the two-service app

Estimated time	Difficulty	Environment	Kit
25 min	...	Cloud Shell, region asia-south1	gcp-landscape-lab5-starter.zip (solution: -solution.zip)

OBJECTIVE

The supplied Bookshelf app — bookshelf-api (internal, authenticated) and bookshelf-web (public) — built by Cloud Build into Artifact Registry and running on Cloud Run with their own service accounts, web reaching api through the VPC.

PREREQUISITES

- Labs 0 and 2 completed (Lab 4 helps: Private Google Access appears again).
- Kit unzipped: api/ and web/ (both complete — nothing to edit), deploy.sh with four blanks, teardown.sh.

Both services are supplied finished, Dockerfile included. Every step is a gcloud command or a console setting. Run everything from the lab5 folder.

Steps

Step 0 — Optional — see the app run locally (two Cloud Shell tabs)

CLOUD SHELL

```
cd api && pip install -q -r requirements.txt && PORT=8091 python app.py      # tab 1
cd web && pip install -q -r requirements.txt && PORT=8090 python app.py      # tab 2
curl -s localhost:8090/ | grep -o "<li>.*</li>" | head -4
```

EXPECTED

```
<li><b>The Pragmatic Programmer</b> — Hunt & Thomas (1999)</li>
...four rows
```

Locally the web service needs no token; on Cloud Run it will mint one automatically — that is already in the code.

Step 1 — Fill blank 5.1 — a Docker repository in Artifact Registry

Console path: Artifact Registry → Repositories → Create → name bookshelf, format Docker, mode Standard, region asia-south1.

CLOUD SHELL

```
REGION=asia-south1; PROJECT=$(gcloud config get-value project); REPO=bookshelf
AR=$REGION-docker.pkg.dev/$PROJECT/$REPO
gcloud config set run/region $REGION
gcloud artifacts repositories create $REPO --repository-format=docker \
  --location=$REGION --description="Bookshelf images"
```

EXPECTED

```
Created repository [bookshelf].
```

Step 2 — Build both images with Cloud Build and push them

No Docker on your machine: the Dockerfile runs on a Cloud Build worker and the image lands in the repo.

Console path: Cloud Build → History shows each build; Artifact Registry → bookshelf lists the images.

CLOUD SHELL

```
gcloud builds submit api --tag $AR/bookshelf-api:1.0
gcloud builds submit web --tag $AR/bookshelf-web:1.0
gcloud artifacts docker images list $AR
```

EXPECTED

Creating temporary archive of 4 file(s) totalling 2.1 KiB before compression.

...

Step #0: Step 1/9 : FROM python:3.13-slim

...

1.0: digest: sha256:4b7c3e2f... size: 1786

ID	CREATE_TIME	DURATION	SOURCE	IMAGES
7f3e...	2026-09-06T10:25:12+00:00	1m4s	gs://.../source.tgz	asia-south1-
docker.pkg.dev/.../bookshelf-api:1.0		SUCCESS		

IMAGE	CREATE_TIME	UPDATE_TIME	DIGEST
asia-south1-docker.pkg.dev/sandbox-rohan-07/bookshelf/bookshelf-api	2026-09-06T10:26:15	2026-09-06T10:26:15	sha256:4b7c3e2f...
asia-south1-docker.pkg.dev/sandbox-rohan-07/bookshelf/bookshelf-web	2026-09-06T10:27:31	2026-09-06T10:27:31	sha256:9e1d5c7a...

About a minute per image.

Step 3 — One dedicated identity per service — with no roles at all

Console path: IAM & Admin → Service accounts → Create (twice): bookshelf-api-sa, bookshelf-web-sa. Grant no roles.

CLOUD SHELL

```
gcloud iam service-accounts create bookshelf-api-sa --display-name="bookshelf-api
runtime"
gcloud iam service-accounts create bookshelf-web-sa --display-name="bookshelf-web
runtime"
```

EXPECTED

Created service account [bookshelf-api-sa].
Created service account [bookshelf-web-sa].

Step 4 — Fill blank 5.4 — deploy the api: internal ingress, callers must authenticate

Console path: Cloud Run → Deploy container → Service → image bookshelf-api:1.0 → Security: Require authentication, service account bookshelf-api-sa → Networking: Ingress Internal → Create.

CLOUD SHELL

```
gcloud run deploy bookshelf-api --image $AR/bookshelf-api:1.0 \
  --service-account bookshelf-api-sa@$PROJECT.iam.gserviceaccount.com \
  --ingress internal --no-allow-unauthenticated
API_URL=$(gcloud run services describe bookshelf-api --format='value(status.url)')
curl -s -o /dev/null -w "from Cloud Shell: HTTP %{http_code}\n" $API_URL/
```

EXPECTED

Service [bookshelf-api] revision [bookshelf-api-00001-mz4] has been deployed and is serving 100 percent of traffic.
Service URL: https://bookshelf-api-482915730261.asia-south1.run.app
from Cloud Shell: HTTP 404

404 is the success signal: Cloud Shell's request arrives from the internet and internal ingress hides the service from it.

Step 5 — Fill blank 5.5 — let ONLY the web identity call the api

Console path: Cloud Run → bookshelf-api → Permissions → Add principal → bookshelf-web-sa@... → role Cloud Run Invoker → Save.

CLOUD SHELL

```
gcloud run services add-iam-policy-binding bookshelf-api \
  --member=serviceAccount:bookshelf-web-sa@$PROJECT.iam.gserviceaccount.com \
  --role=roles/run.invoker
```

EXPECTED

```
Updated IAM policy for service [bookshelf-api].
bindings:
- members:
  - serviceAccount:bookshelf-web-sa@sandbox-rohan-07.iam.gserviceaccount.com
  role: roles/run.invoker
```

CHECK YOURSELF

- Two images in the repo, two service accounts with no project roles.
- The api answers 404 from Cloud Shell.
- The api's IAM policy lists exactly one invoker: bookshelf-web-sa.

Step 6 — Fill blank 5.6 — route web's egress through the VPC and deploy it

To reach an internal-ingress service, the caller's traffic must arrive through a VPC: Direct VPC egress with all traffic routed, plus Private Google Access on that subnet.

Console path: VPC network → default → Subnets → asia-south1 → Edit → Private Google Access On. Then Cloud Run → Deploy container → image bookshelf-web:1.0 → Allow unauthenticated → service account bookshelf-web-sa → Variables: API_URL → Networking: Connect to a VPC for outbound traffic → Send all traffic to the VPC → Create.

CLOUD SHELL

```
gcloud compute networks subnets update default --region=$REGION --enable-private-ip-
google-access
gcloud run deploy bookshelf-web --image $AR/bookshelf-web:1.0 \
  --service-account bookshelf-web-sa@$PROJECT.iam.gserviceaccount.com \
  --network default --subnet default --vpc-egress all-traffic \
  --set-env-vars API_URL=$API_URL \
  --allow-unauthenticated
WEB_URL=$(gcloud run services describe bookshelf-web --format='value(status.url)')
echo "open: $WEB_URL"
curl -s $WEB_URL/ | grep -o "<li>.*</li>" | head -4
```

EXPECTED

```
Service URL: https://bookshelf-web-482915730261.asia-south1.run.app
<li><b>The Pragmatic Programmer</b> - Hunt & Thomas (1999)</li>
<li><b>Designing Data-Intensive Applications</b> - Kleppmann (2017)</li>
<li><b>Site Reliability Engineering</b> - Beyer et al. (2016)</li>
<li><b>Release It!</b> - Nygard (2018)</li>
```

Open the URL in a browser: the page lists four books and names the api URL and version. Same api URL, two answers — the path through the network decided who got in.

Step 7 — Tear down everything

Also removes Lab 2's cloud-run-source-deploy repository.

CLOUD SHELL

```
bash teardown.sh
gcloud run services list; gcloud artifacts repositories list --location=$REGION
```

EXPECTED

```
Lab 5 resources removed. Run the Lab 1/3/4 teardown scripts too if you skipped them.
Listed 0 items.
Listed 0 items.
```

Troubleshooting

Symptom	Cause	Fix
Page says "could not reach bookshelf-api: 403"	run.invoker binding missing or on the wrong service account	Re-run step 5 with bookshelf-web-sa
Page says "... 404"	web's egress is not going through the VPC, or PGA is off	Check --network/--subnet/--vpc-egress all-traffic and step 6's subnet update
Page says connection error	API_URL env var missing on web	Re-run step 6; check \$API_URL is set in this shell
deploy: service account does not exist	IAM propagation delay (~30 s)	Wait and re-run the deploy
builds submit: permission denied	cloudbuild API not enabled or missing role	Lab 0 step 3; the deployer needs roles/cloudbuild.builds.editor (Owner has it)

Challenge (fast finishers)

Add a second revision of the api with an image tag 1.1 (rebuild with --tag \$AR/bookshelf-api:1.1 and deploy with --no-traffic --tag canary), then move 10 % of traffic to it with update-traffic. Nothing about web changes. Then place every resource you created on the landscape diagram from the deck's recap slide.

Summary

Build once into the registry, run each service as its own identity, let exactly one caller in, and reach a private service only through the VPC. You never created a key, a password, a public VM IP or a cluster — and never wrote a line of code.

Activities A and B (no keyboard)

Activity A — The decision matrix

Format: pairs, about 12 minutes. Copy the grid below into a notebook. For each workload card, tick VM / GKE / Cloud Run-or-function per criterion, then write the decision and the one criterion that decided it.

Criterion	Card 1 · Order-export job	Card 2 · Licensed reporting engine	Card 3 · Public catalogue API	Card 4 · Bookshelf web + api
1 State	stateless — CR job ✓ (worked example)			
2 Traffic shape				
3 Latency budget				
4 Control needed				
5 Team				
6 Portability				
Decision + deciding criterion				

Card 1 — runs nightly at 02:00 for ~15 min; reads the orders DB; writes a CSV to a bucket. Card 2 — vendor binary needing a kernel module and a static licence IP; 20 analysts 9–6. Card 3 — 0 req/s at night, 400 req/s at 9 am; < 300 ms; stateless; no ops team. Card 4 — two small stateless Python services; web public, api reachable by web only; the team is this room.

MODEL ANSWERS (TRAINER RELEASES AFTER THE DEBRIEF)

- Card 1 → Cloud Run job on Cloud Scheduler — traffic shape decides.
- Card 2 → Compute Engine VM in a MIG of 1 — control decides.
- Card 3 → Cloud Run service with min-instances 1–2 — latency and team decide.
- Card 4 → two Cloud Run services, api internal + IAM-only — state and team decide (this is Lab 5).

Activity B — Pick the store

Format: individually, then compare with a neighbour, about 6 minutes. Use the decision tree from the deck: files or records → transactional or analytical → relational or not → one region or many.

#	Workload	Your pick	Why (one line)
1	Product images uploaded by sellers, served on every page view, kept forever		
2	Orders and payments for one country; strict consistency; 2 000 writes/s at peak		
3	"Revenue by category by hour, last 3 years" for the finance dashboard		
4	Per-user shopping carts with live updates on web and mobile		
5	Bookshelf's catalogue — 4 books today, maybe 4 000 next year; read by the api on every request		

MODEL ANSWERS

- 1 → Cloud Storage, Standard (multi-region if global) + Cloud CDN
- 2 → Cloud SQL (PostgreSQL); Spanner only for multi-region writes
- 3 → BigQuery
- 4 → Firestore
- 5 → Cloud SQL (+ Memorystore later); today's in-memory list is the placeholder

Appendix — Environment and versions

Item	Pinned	Note
gcloud CLI	583.x	Cloud Shell keeps itself current; re-verify with gcloud --version
Region / zone	asia-south1 / asia-south1-a	Mumbai — all labs
VM image	debian-12 (debian-cloud)	image family, not a dated image
Machine types	e2-micro (VMs), e2-small (MIG template)	quota-friendly
Python (sample app)	3.13 — python313 base image, python:3.13-slim Docker	supplied complete; not edited
Flask / gunicorn / functions-framework	3.1.3 / 26.2.0 / 3.10.2	pinned in the kits
Registry	Artifact Registry only	Container Registry (gcr.io) shut down 18 Mar 2025
Cloud Run functions	gcloud run deploy --function	Cloud Functions is now Cloud Run functions

APIs enabled in Lab 0

- compute.googleapis.com
- run.googleapis.com
- cloudbuild.googleapis.com
- artifactregistry.googleapis.com
- secretmanager.googleapis.com
- storage.googleapis.com

Kits

Lab	Starter	Solution
1	gcp-landscape-lab1-starter.zip — lab1.sh (3 blanks), startup.sh, teardown.sh	gcp-landscape-lab1-solution.zip — completed lab1.sh
2	gcp-landscape-lab2-starter.zip — bookshelf-api/, hello-function/, lab2.sh (4 blanks), teardown.sh	completed lab2.sh
3	gcp-landscape-lab3-starter.zip — docs/, lifecycle.json (3 blanks), lab3.sh, teardown.sh	completed lifecycle.json
4	gcp-landscape-lab4-starter.zip — lab4.sh (4 blanks), teardown.sh	completed lab4.sh
5	gcp-landscape-lab5-starter.zip — api/, web/, deploy.sh (4 blanks), teardown.sh	completed deploy.sh

End-of-session teardown order

- `bash lab5/teardown.sh`
- `bash lab4/teardown.sh`
- `bash lab3/teardown.sh`

- `bash lab2/teardown.sh` and `bash lab1/teardown.sh` if skipped earlier
- Verify: `gcloud compute instances list` · `gcloud run services list` · `gcloud storage ls` · `gcloud compute networks list` → only 'default' remains

Volatile facts (versions, product names, console navigation) were verified against docs.cloud.google.com on 6 September 2026. Re-verify before each delivery.